

D6T-8L-09H Temperature Sensor

D6T-8L-09H Temperature Sensor

Resource: [User's Manual](#)

(Tested Successfully)

CONNECTIONS and SPECIFICATIONS:

4 Usage Procedure

4.1 Connectors

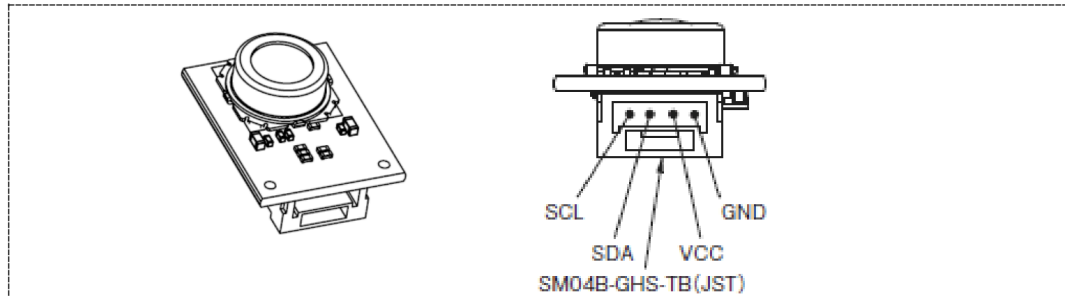


Fig. 4. Product Exterior (Reference)

Connector Pins

Table 1. Connector Pin Table

1	GND	GND power supply pin
2	VCC	VCC power supply pin (5 V $\pm$ 10%)
3	SDA	I ² C (5 V) data
4	SCL	I ² C (5 V) clock

FAQs:

•D6T-8L-09H

- Are the data measured at intervals of 250 ms or longer?
- Is the initialization command sent after the power is turned on?
- Is the sensor data converted correctly? (D6T-8L-09H outputs a value 5 times the temperature.)

Example) Temperature output value: 560 -> $560/5 = 112.0$ (°C)

PTAT: Proportional To Absolute Temperature

The D6T uses infrared thermopiles

Each pixel senses IR radiation and converts it into temperature — but those readings depend slightly on the **sensor's internal temperature**.

So PTAT helps us:

Know if the sensor got warm due to self-heating

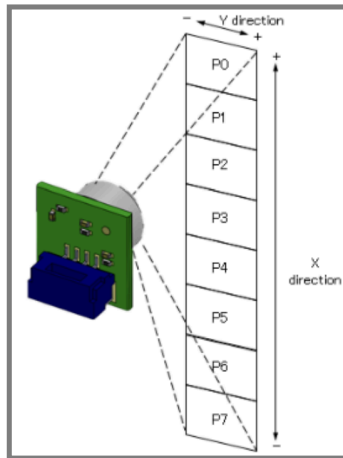
Optionally correct pixel readings if you need high accuracy (Omron does this internally in their high-end sensors).

[Received data item]

PTAT: Reference temperature data inside the sensor. 10-fold temperature value [°C] with a 16-bit signed integer (two's complement).

(Calculation example)

When PTAT (Low): 0x20, PTAT (High): 0x01, 0x0120 -> 288 -> $288/10 = 28.8$ (°C) at Int16



P0 to P7: Temperature data for each pixel. 5-fold temperature value [°C] with a 16-bit signed integer (two's complement)

(Calculation example)

When P0 (Low): 0x30, P0 (High): 0x02, 0x0230 -> 560 -> $560/5 = 112.0$ (°C) at Int16

This does not include the use of I2C switch

The structure of the sample code is as follows.

```
/** <!-- main - Thermal sensor {{{1 -->
 * 1. Initialize.
 * 2. Read data
 */
int main() {
    int i;
    int16_t itemp;

    delay(20);
    // 1. Initialize
    InitialSetting();
    delay(500);
```

```
void InitialSetting(void) {
    uint8_t dat1[] = {0x02, 0x00, 0x01, 0xee};
    i2c_write_reg8(D6T_ADDR, dat1, sizeof(dat1));
    uint8_t dat2[] = {0x05, 0x90, 0x3a, 0xb8};
    i2c_write_reg8(D6T_ADDR, dat2, sizeof(dat2));
    uint8_t dat3[] = {0x03, 0x00, 0x03, 0x8b};
    i2c_write_reg8(D6T_ADDR, dat3, sizeof(dat3));
    uint8_t dat4[] = {0x03, 0x00, 0x07, 0x97};
    i2c_write_reg8(D6T_ADDR, dat4, sizeof(dat4));
    uint8_t dat5[] = {0x02, 0x00, 0x00, 0xe9};
    i2c_write_reg8(D6T_ADDR, dat5, sizeof(dat5));
}
```

```
while(1){
    // 2. Read data
    // Read data via I2C
    memset(rbuf, 0, N_READ);
    uint32_t ret = i2c_read_reg8(D6T_ADDR, D6T_CMD, rbuf, N_READ);
    D6T_checkPEC(rbuf, N_READ - 1);

    //Convert to temperature data (degC)
    ptat = (double)convBus_s16_le(rbuf, 0) / 10.0;
    for (i = 0; i < N_PIXEL; i++) {
        itemp = convBus_s16_le(rbuf, 2 + 2*i);
        pix_data[i] = (double)itemp / 5.0;
    }

    //Output results
    printf("PTAT: %4.1f [degC], Temperature: ", ptat);
    for (i = 0; i < N_PIXEL; i++) {
        printf("%4.1f, ", pix_data[i]);
    }
    printf("[degC]\n");

    delay(250);
}
}
```

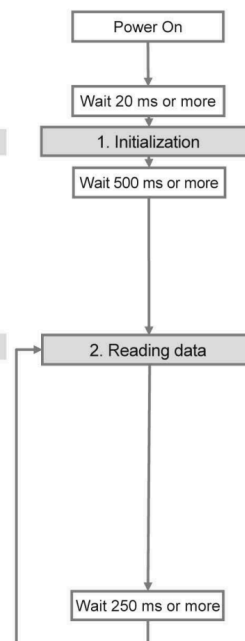
I2C Read
PEC confirmation

Convert to PTAT
Convert to temperature value

Display on screen

1

2



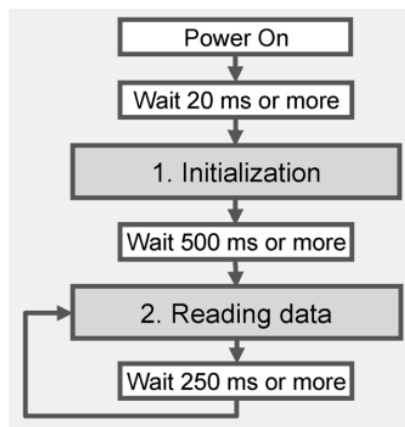
(1) Outline of I²C Interface

Slave Address	7bit (0001_010b) 8bit (with R/W bit) expression: Read : 15h, Write : 14h
Data bit width	8bit (MSB-first)
Clock speed	Max. 100kHz
Clock stretch supported	Not supported

(2) Communication procedure

The procedure for communicating with the sensor is as shown below.

The sensor with standard specifications updates the measurement data within every 250 ms. This is repeated regardless of the communication, and the measurement timing cannot be controlled externally.

1. Initialization

Send and receive commands as shown below.

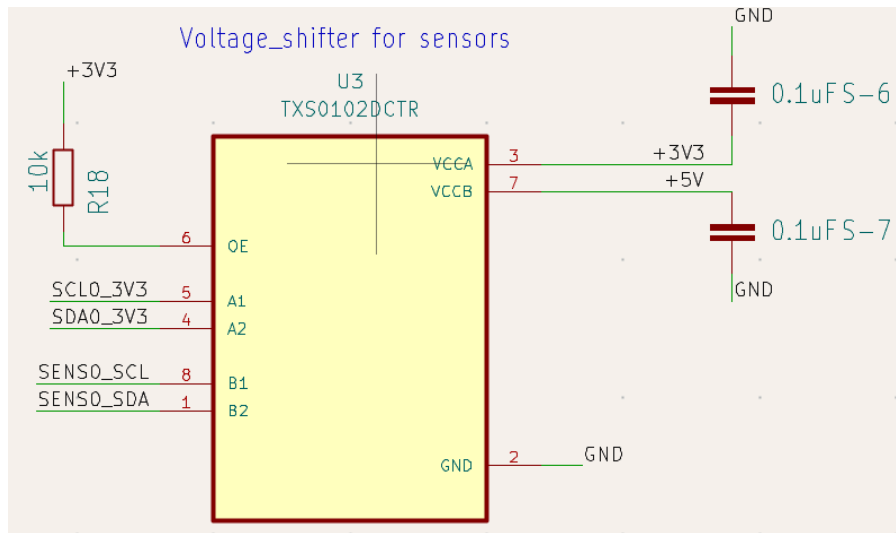
Slave Address + Write

Start	0x14	0x02	0x00	0x01	0xEE	Stop
Start	0x14	0x05	0x90	0x3A	0xB8	Stop
Start	0x14	0x03	0x00	0x03	0x8B	Stop
Start	0x14	0x03	0x00	0x07	0x97	Stop
Start	0x14	0x02	0x00	0x00	0xE9	Stop

Slave Address + Read						
Start	0x14	0x02	Repeat Start	0x15	Should be 0x00	Should be 0x00
Start	0x14	0x05	Repeat Start	0x15	Should be 0x90	Should be 0x3A
Start	0x14	0x03	Repeat Start	0x15	Should be 0x00	Should be 0x07

Check if written
correctly
(optional)

VOLTAGE SHIFTERS:



We are required to use a voltage shifter because the sensors work at 5V but teensy requires 3v3 signals

We receive B1 and B2 from the sensors

Scenario 3: Using an I²C Level Converter

(Not a 5 V fault tolerant specification, or other devices are also connected to the 3 V I²C bus)

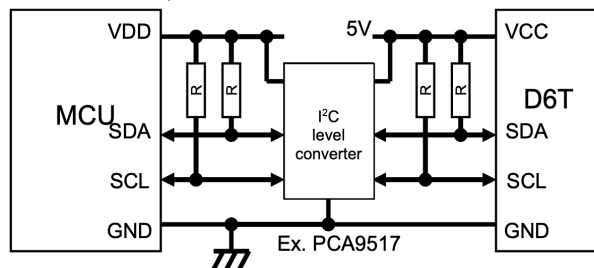


Fig. 7. Using a Level Converter

A1 and A2 go to the teensy

2 separate voltage level converters are used for the two sensors used.

LOGICAL FLOW:

Sensor1 -> V_shift ----|
 Sensor2 -> V_shift ----|
 —> I2C Switch —> Teensy

I2C SWITCH:

Scenario 5: Using an I²C Bus-Switching IC (Connecting multiple D6T sensors)
(This sensor cannot change slave addresses)

*1. Most bus-switching ICs also have power voltage conversion functionality.

We are using a 2 channel I2C switch. This is because the address of both the slaves (temperature sensors) is the same and cannot be changed by us.

We can choose which sensor to receive the data from via the code if we use a I2C switch instead.

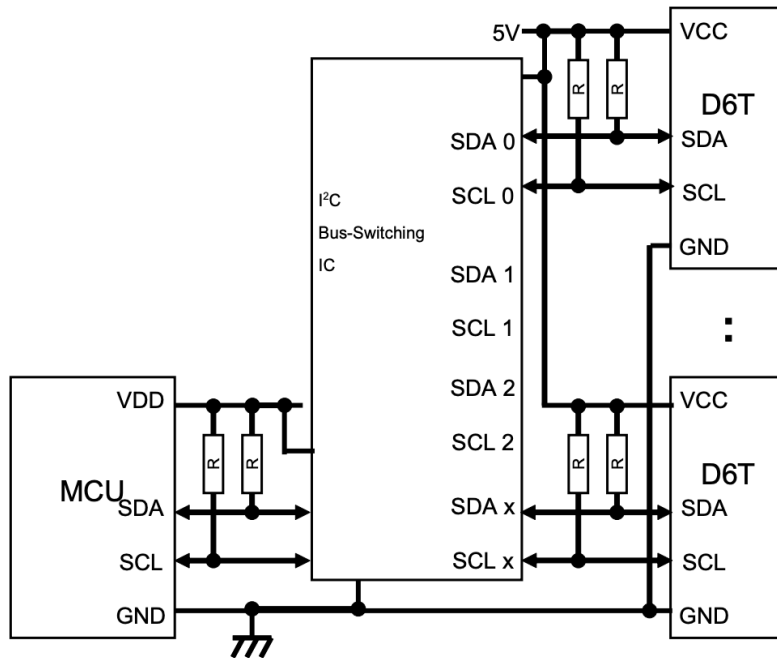
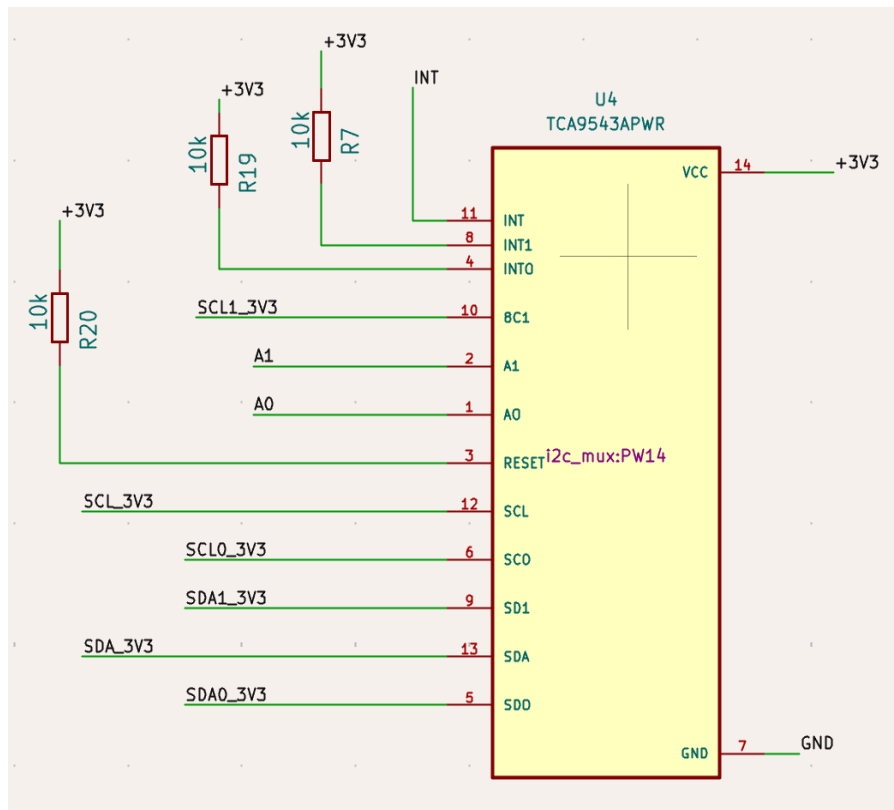


Fig. 9. When using the I²C Bus-Switching IC



Code


```

#include <Arduino.h>
#include <Wire.h>

#define TCA9543A_ADDR 0x70 // I2C switch default address
#define D6T_ADDR 0x0A      // Omron D6T-8L-09H fixed address

// CRC-8 function
uint8_t calcCRC8(uint8_t *data, int len)
{
    uint8_t crc = 0;
    for (int i = 0; i < len; i++)
    {
        crc ^= data[i];
        for (uint8_t b = 0; b < 8; b++)
        {
            crc = (crc & 0x80) ? (crc << 1) ^ 0x07 : (crc << 1);
        }
    }
    return crc;
}

// Select channel on TCA9543A (0 or 1)
void selectChannel(uint8_t channel)
{
    Wire.beginTransmission(TCA9543A_ADDR);
    Wire.write(1 << channel); // channel 0 = 0x01, channel 1 = 0x02
    Wire.endTransmission();
    delay(5);
}

// Read D6T-8L-09H data
bool readD6T(int16_t *ptat, int16_t *pixels, uint8_t numPixels = 8)
{
    Wire.beginTransmission(D6T_ADDR);
    Wire.write(0x4C);
    if (Wire.endTransmission(false) != 0)
        return false;

    delay(20); // Specified in user manual

    int bytesToRead = 2 + 2 * numPixels + 1;
    int received = Wire.requestFrom(D6T_ADDR, bytesToRead);

```

```

    if (received < bytesToRead)
        return false;

    uint8_t buf[32];
    for (int i = 0; i < bytesToRead; i++)
        buf[i] = Wire.read();

    if (calcCRC8(buf, bytesToRead - 1) != buf[bytesToRead - 1])
        return false;

    *ptat = (int16_t)((buf[1] << 8) | buf[0]);
    for (uint8_t i = 0; i < numPixels; i++)
    {
        uint8_t l = buf[2 + i * 2];
        uint8_t h = buf[3 + i * 2];
        pixels[i] = (int16_t)((h << 8) | l);
    }
    return true;
}

void setup()
{
    Serial.begin(115200);
    Wire.begin();
    Wire.setClock(400000);
    delay(1000);
    Serial.println("D6T-8L-09H + TCA9543A on Teensy 4.0");
    Serial.println("channel,ptat,pix0,pix1,...,pix7");
}

void loop()
{
    int16_t ptat_raw;
    int16_t pixels_raw[8];

    for (uint8_t ch = 0; ch < 2; ch++)
    {
        selectChannel(ch);

        if (readD6T(&ptat_raw, pixels_raw))
        {
            float ptat_c = ptat_raw / 10.0f;

```

```
Serial.print(ch);  
Serial.print(",");  
Serial.print(ptat_c, 1);  
for (uint8_t i = 0; i < 8; i++)  
{  
    float temp_c = pixels_raw[i] / 5.0f;  
    Serial.print(",");  
    Serial.print(temp_c, 1);  
}  
Serial.println();  
}  
else  
{  
    Serial.print("Failed to read sensor on channel ");  
    Serial.println(ch);  
}  
}  
delay(500);  
}
```

Code_Final

```

/*
 * This sketch initializes two D6T sensors on different I2C channels,
 * then continuously reads the ambient temperature (PTAT) and the 8 pixel
 * temperatures from each sensor, printing the data to the Serial Monitor
 * in CSV format.
 *
 * Connections:
 * - Arduino SDA -> TCA9543A SDA
 * - Arduino SCL -> TCA9543A SCL
 * - TCA9543A SD0/SC0 -> D6T Sensor 1 SDA/SCL
 * - TCA9543A SD1/SC1 -> D6T Sensor 2 SDA/SCL
 * - Power and Ground for all devices.
 */
#include <Arduino.h>
#include <Wire.h>

// Addresses
#define TCA9543A_ADDR 0x70 // I2C switch
#define D6T_ADDR 0x0A      // D6T-8L-09H 7-bit address
#define D6T_CMD 0x4C       // Measurement command

// D6T Sensor Settings
#define N_PIXEL 8
#define N_READ ((N_PIXEL + 1) * 2 + 1) // PTAT + pixels + PEC

uint8_t rbuf[N_READ];
double ptat;
double pix_data[N_PIXEL];

// CRC / PEC calculation
uint8_t calc_crc(uint8_t data)
{
    for (int i = 0; i < 8; i++)
    {
        uint8_t temp = data;
        data <<= 1;
        if (temp & 0x80)
            data ^= 0x07;
    }
    return data;
}

```

```

}

bool D6T_checkPEC(uint8_t buf[], int n)
{
    uint8_t crc = calc_crc((D6T_ADDR << 1) | 1); // I2C Read address (8-bit)
    for (int i = 0; i < n; i++)
        crc = calc_crc(buf[i] ^ crc);
    bool ret = (crc != buf[n]);
    if (ret)
    {
        Serial.print("PEC check failed: ");
        Serial.print(crc, HEX);
        Serial.print(" (cal) vs ");
        Serial.println(buf[n], HEX);
    }
    return ret;
}

// Utility
int16_t conv8us_s16_le(uint8_t *buf, int n)
{
    uint16_t ret = (uint16_t)buf[n] | ((uint16_t)buf[n + 1] << 8);
    return (int16_t)ret;
}

// Multiplexer Channel Select
void selectChannel(uint8_t ch)
{
    Wire.beginTransmission(TCA9543A_ADDR);
    Wire.write(1 << ch); // 0x01 for channel 0, 0x02 for channel 1
    Wire.endTransmission();
    delay(5);
}

// D6T Initialization
void initD6T()
{
    // Initialization sequence per Omron documentation
    Wire.beginTransmission(D6T_ADDR);
    Wire.write(0x02);
    Wire.write(0x00);
    Wire.write(0x01);
}

```

```
Wire.write(0xEE);
Wire.endTransmission();

Wire.beginTransmission(D6T_ADDR);
Wire.write(0x05);
Wire.write(0x90);
Wire.write(0x3A);
Wire.write(0xB8);
Wire.endTransmission();

Wire.beginTransmission(D6T_ADDR);
Wire.write(0x03);
Wire.write(0x00);
Wire.write(0x03);
Wire.write(0x8B);
Wire.endTransmission();

Wire.beginTransmission(D6T_ADDR);
Wire.write(0x03);
Wire.write(0x00);
Wire.write(0x07);
Wire.write(0x97);
Wire.endTransmission();

Wire.beginTransmission(D6T_ADDR);
Wire.write(0x02);
Wire.write(0x00);
Wire.write(0x00);
Wire.write(0xE9);
Wire.endTransmission();

delay(500);
}

// D6T Data Read
bool readD6T()
{
    memset(rbuf, 0, N_READ);
    Wire.beginTransmission(D6T_ADDR);
    Wire.write(D6T_CMD);
    Wire.endTransmission();
```

```

Wire.requestFrom(D6T_ADDR, N_READ);
int i = 0;
while (Wire.available() && i < N_READ)
    rbuf[i++] = Wire.read();

if (D6T_checkPEC(rbuf, N_READ - 1))
    return false;

ptat = conv8us_s16_le(rbuf, 0) / 10.0; // divide by 10 (given in datasheet)
for (int j = 0; j < N_PIXEL; j++)
{
    int16_t itemp = conv8us_s16_le(rbuf, 2 + 2 * j);
    pix_data[j] = itemp / 5.0;
}
return true;
}

// Arduino Setup
void setup()
{
    Serial.begin(115200);
    Wire.begin();
    Wire.setClock(100000); // per D6T spec (max 100 kHz)
    delay(1000);

    Serial.println("D6T-8L-09H Dual Sensor via TCA9543A");
    Serial.println("channel, ptat, pix0, pix1, ..., pix7");

    // Initialize both sensors through TCA9543A
    for (uint8_t ch = 0; ch < 2; ch++)
    {
        selectChannel(ch);
        initD6T();
    }
}

// Main Loop
void loop()
{
    for (uint8_t ch = 0; ch < 2; ch++)
    {
        selectChannel(ch);
    }
}

```



```
if (readD6T())
{
    Serial.print(ch);
    Serial.print(", ");
    Serial.print(ptat, 1);
    for (uint8_t i = 0; i < N_PIXEL; i++)
    {
        Serial.print(", ");
        Serial.print(pix_data[i], 1);
    }
    Serial.println();
}
else
{
    Serial.print("Failed to read sensor on channel ");
    Serial.println(ch);
}
}
delay(500);
}
```